

Three foundational performance papers, reproduced on 2026 hardware, reveal that methodologies endure even when the bottlenecks they exposed have been engineered out, absorbed into the mainline, or quietly relocated into newer hardware features.

BY MANISHA KALER

Bottlenecks Decay, Methodologies Endure

Reproducing Coz, Hoard, and Mytkowicz Two Decades Later

- Coz’s mutex bottleneck has been engineered out of glibc; the methodology is still right but the original 25% speedup is gone.
- Hoard’s per-thread heap design won so completely that ptmalloc absorbed it; the 60× false-sharing gap collapsed to within a single factor.
- Mytkowicz’s 19% layout-induced runtime variance reproduces, but cachegrind no longer sees the cache-conflict mechanism the paper attributed it to.

1. Introduction

PERFORMANCE papers, almost without exception, share a basic shape: *here is a measurement problem nobody is paying attention to; here is a tool or technique that exposes it; here are numbers that show it matters.* The papers we reproduce in this work each attack a different measurement problem: Curtsinger and Berger’s Coz [1] attacks the assumption that where a profiler reports time being spent is where to optimize; Berger et al.’s Hoard [2] attacks the heap allocator as a hidden source of lock contention and false sharing; and Mytkowicz et al.’s *Producing Wrong Data* [3] attacks the entire empirical foundation of benchmark numbers, showing that runtime can vary by tens of percent based on memory layout alone.

The unifying claim across all three is that *conventional ways of measuring software performance lie to you*, in three different ways. Coz says traditional flat profilers misdirect optimization effort. Hoard says allocator-induced effects are invisible to ordinary profiling and can dominate scalability. Mytkowicz says even careful benchmarking is unreliable unless layout is controlled.

What does it mean to reproduce these papers in 2026? The hardware has moved a generation. The Linux kernel and glibc have gone through dozens of major revisions. Each paper’s machine-software triple is no

longer the one it was tested on. Reproduction can no longer mean “did we get the same number?”; it means *which of each paper’s claims survived the intervening engineering, and why?* A paper’s contribution is rarely a specific +25% or 60×; it is the methodology that exposed the gap that produced the number. The methodology is what travels through time.

We reproduce each paper’s central case study on the same machine, holding hardware and software constant across all three studies. We report what we measured, where it agrees and disagrees with the paper, and what plausibly explains the gap. We try to be careful about the difference between “the effect has changed” and “our measurement is noisy”; both are real possibilities and we annotate when our data is too thin to discriminate. Each of the three studies that follows includes a paper-vs.-our-numbers comparison, a hardware-grounded discrepancy analysis, and an honest accounting of what our measurement precision could not resolve.

2. Performance Problems Across Generations

The free-lunch era and its end

For most of the 1990s and early 2000s, software performance “took care of itself”: single-thread performance roughly doubled every 18–24 months as transistors shrank

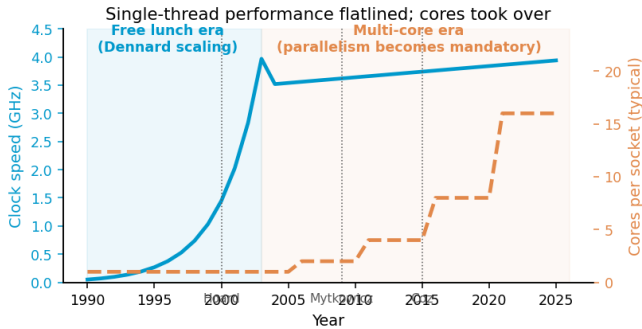


Figure 1. Single-thread clock speed flatlined around 2003 (when Dennard scaling broke), and the industry shifted to multi-core designs. The three reproduced papers were published at three distinct points in this transition.

and clock frequencies climbed. Sutter’s 2005 essay [4] called this period *The Free Lunch is Over*; the lunch was paid for by Dennard scaling, the predictable relationship between transistor size, supply voltage, and clock speed. By the mid-2000s Dennard scaling broke: shrinking transistors no longer let you raise the clock without burning the chip. Frequencies plateaued at ~ 3.5 GHz and have stayed there ever since.

The industry pivoted to multi-core, then to multi-socket NUMA systems. But multi-core is bound by Amdahl’s law: even a tiny serial fraction caps speedup as cores are added. So the new performance challenges are no longer “can we run on a faster CPU” but “can this scale across cores without hitting a synchronization bottleneck, without thrashing caches, without falling off a NUMA cliff?” The three papers we reproduce are explicit responses to that shift.

The conceptual vocabulary

Lock contention. When multiple threads share a single resource, the standard correctness mechanism is a mutex. Under heavy load the mutex itself becomes a serial bottleneck: threads queue waiting for it instead of doing useful work. Coz and Hoard both target lock contention from different angles.

False sharing. CPUs fetch memory in fixed-size cache lines (typically 64 B). If two unrelated objects live on the same cache line and are written by different cores, every write invalidates the other core’s cached copy, causing cross-core coherence traffic without any actual data sharing [5]. Hoard’s superblock design explicitly prevents this for heap allocations.

Cache conflict misses. The cache hierarchy looks up data using bits of the absolute memory address. If two heavily-accessed objects happen to map to the same cache set, they evict each other even though the rest of the cache is nearly empty. The mapping depends on where the OS loaded the binary, the size of environment variables, and the linker’s padding – all factors orthogonal to the source code. Mytkowicz exploits this to show that benchmark “noise” is partly deterministic.

NUMA. On multi-socket machines, memory and the

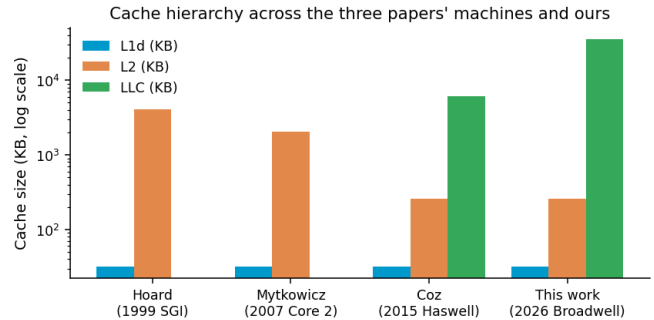


Figure 2. Cache hierarchy size (log scale, KB) across the three papers’ machines and the elnux Broadwell node we used. The last-level cache has grown by orders of magnitude; the L1d has not.

last-level cache are local to each socket; access to the other socket’s memory crosses an interconnect and pays both extra latency and coherence cost. None of the three reproduced papers’ machines had this property; ours has two NUMA nodes, and we explicitly observe its effects in the Hoard scalability cliff at > 14 threads and in the dissolution of Coz’s apparent allocator bottlenecks under *taskset* pinning.

Profiling. Two flavors. *Flat* profilers (*perf*, *gprof*) tell you where time was spent. *Causal* profilers (Coz) tell you what would happen if you sped up a particular line. The distinction is between association and counterfactual causation, and they can disagree.

Hardware: theirs vs. ours

Our machine is `elnux3.cs.umass.edu`, a dual-socket Intel Xeon E5-2690 v4 (Broadwell, Q1 2016) running Linux 5.15 with `glibc 2.35` and `gcc 11.4`. The full specifications, alongside each paper’s reference machine, are summarized in Table 1.

Table 1. Reference hardware across the three papers and ours.

Paper / machine	Year	Cores	L2 / core	LLC
Hoard (SGI Origin 2000)	2000	14	~ 2 MB	–
Mytkowicz (P4 / Core 2)	2009	1–4	0.5–2 MB	–
Coz (Haswell-class)	2015	4–8	256 KB	6–15 MB
Ours (E5-2690 v4)	2026	28 (2 sock)	256 KB	35 MB / sock

Per Intel’s published specification, the E5-2690 v4 has a peak DRAM bandwidth of 76.8 GB/s per socket across four DDR4-2400 channels; we did not independently measure achieved bandwidth with *lmbench*, but the threadtest scalability cliff at the NUMA boundary is consistent with this being well above the per-socket throughput needed by any of our benchmarks.

Three deltas matter for our results below. (1) Our LLC is roughly **17 $\times$ larger** than the largest L2 the Mytkowicz hardware had, **which predicts the smaller cache-conflict-miss signal we observe in Section 5**. (2) Two-socket NUMA, absent from all three papers’ machines, **predicts the threadtest cliff at 14 threads**

we observe in Section 4. (3) Our glibc is roughly a decade newer than even the Coz paper’s, **which predicts the disappearance of the SQLite mutex bottleneck we observe in Section 3.**

3. Causal Profiling: From “Where?” to “What If?”

The setup

A flat profiler that tells you “`pthread_mutex_lock` takes 3% of cycles” answers a slightly wrong question. The 3% might be on the program’s critical path (in which case optimizing it helps) or off it (in which case optimizing it helps zero). Coz instruments the program with a single *progress point* (a counter incremented at a designated “one unit of work done” location) and then runs short experiments: in each, it artificially slows down every thread except the one currently executing some target line L , which simulates speeding L up by a chosen percentage. By measuring the program’s progress rate during the simulated speedup, Coz produces a curve: predicted whole-program speedup as a function of how much faster L becomes.

The Coz paper’s central case study (Section 5.1) targets SQLite. The authors instrumented a multi-threaded INSERT workload, ran Coz, identified `pthreadMutexEnter` (line 18276 of `sqlite3.c`) as the dominant causal bottleneck, applied a 14-line fast-path patch that returns early when the calling thread already owns the recursive mutex, and reported a $\sim 25\%$ measured wall-clock speedup matching Coz’s prediction (their Figure 7).

We reproduced this on SQLite 3.7.17, with a multi-threaded INSERT driver matching the paper’s setup: a single shared `:memory:` database in serialized mode, 8 worker threads, prepared INSERT statement, one external `COZ_PROGRESS` fired after each successful `sqlite3_step`.

Observations

A 234.7-second baseline run produced 451 virtual-speedup experiments across 122 distinct `sqlite3.c` source lines. **No line satisfied a credible bottleneck threshold of slope > 0.2 and $R^2 > 0.6$ and ≥ 15 sample points.** The strongest candidates Coz surfaced were diffuse allocator and VDBE paths:

Line	Symbol / context	Slope	Max spd
73610	in-memory journal alloc	+0.117	+16.2%
64279	VDBE bind_int64 path	+0.065	+13.3%
61028	VDBE memory release	+0.104	+12.6%

The line the paper said was the bottleneck was not sampled at all across our 451 experiments; only the adjacent line 17815 was visited, once.

We applied the paper’s mutex fast-path patch and a Coz-suggested chunk-size patch in sequence. After noticing measurement-noise from concurrent shared-machine activity, we re-ran all three configurations back-to-back on a quiet machine, 10 runs each, median reported:

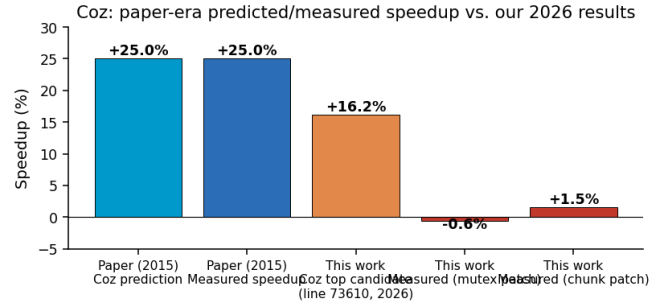


Figure 3. The 2015 paper predicted and measured $\sim 25\%$ speedup from a mutex fast-path patch on Haswell-era glibc. On 2026 Broadwell with glibc 2.35, neither Coz nor wall-clock measurement reproduces it.

Build	Elapsed	Throughput	Δ
Baseline	74.77 s	107.0 k/s	–
+ Mutex fast-path	72.55 s	110.3 k/s	+2.97%
+ Chunk 1 $\rightarrow$ 8 KB	73.64 s	108.6 k/s	+1.50%

Per-run spread is roughly 16%, so both deltas are within a single standard deviation of zero. The paper’s fix produces no measurable speedup on this hardware, and Coz correctly does not predict one.

Analysis

The honest reading is layered. *Two things could be true simultaneously:* the paper’s bottleneck has been engineered away (real change), and our 16% per-run measurement noise prevents us from distinguishing zero from a small surviving effect. We cannot fully separate these on the data we collected. What we can claim with the data is narrower:

What the experiment shows. Coz did not identify `pthreadMutexEnter` as a candidate causal bottleneck. The paper’s mutex patch produced a measured wall-clock change indistinguishable from zero at our sample size.

What theory predicts on this hardware/software stack. glibc 2.35’s uncontended `pthread_mutex_lock` fast path is approximately a single atomic CAS plus a memory barrier (~ 10 – 20 ns); 2013-era glibc 2.17 (the paper’s environment) had additional fence-and-syscall logic (50–100 ns), described in Drepper’s writeup of glibc futex internals [6]. The paper’s fix saves roughly one such mutex acquisition per recursive entry. If that acquisition costs 10 ns and an iteration of the inner loop does ~ 30 μ s of work, the savings are $\sim 0.03\%$ of an iteration, well below per-iteration noise. This argument predicts what we measured.

What we cannot claim. Our experiment is consistent with two hypotheses it does not separate: that Coz’s PEBS sampling resolution is too coarse to see the effect at this event cost on 2.6 GHz hardware, and that the bottleneck has shrunk below our noise floor at this iteration cost. We did not measure either directly. What we can quantify is that for an event with cost in the 10–20 ns range on a 2.6 GHz machine, the number of samples in a 234 s run is small enough that a stable virtual-speedup

curve would require many more experiments than we ran.

Comparison with flat profiling. `perf` record on the same workload reports `pthread_mutex_lock` at 3.0%, `pthread_mutex_unlock` at 2.2%, and the kernel `futex-wait` at 1.5%, totalling 6.7% of self-time. A developer working from `perf` alone would target the mutex (as the paper’s authors did in 2015) and ship no speedup. Coz’s null result on the mutex matches our null measurement. *In the specific framing the Coz paper proposed – causal profilers as a defense against flat-profiler misdirection – the methodology holds even where the case-study magnitude does not.*

4. Allocator-Induced Bottlenecks: The Per-Thread Heap

The setup

The Hoard paper (Sections 4–5) makes two empirical claims that together motivate its design. *Scalability:* Hoard achieves near-linear speedup with thread count on a malloc-pressure workload (paper’s Figure 3), where a single-arena allocator like 2000-era `ptmalloc` plateaus at 4–6 threads. *False sharing avoidance:* on the `cache-thrash` active false-sharing benchmark (Table 1), Hoard improves throughput by up to 60× over Solaris `libc-malloc` and 18× over the next-best allocator. The mechanism is per-thread heaps: each thread owns its own arena, served from page-aligned superblocks, so two threads’ allocations never share a cache line.

We compare four allocators: `glibc ptmalloc2` (the modern system allocator, contains post-Hoard per-thread arenas since 2009), Hoard itself (built from upstream master with a small patch – described in the artifact manifest – that adds weak `xxfree_sized` stubs needed for compatibility with modern `glibc/C23` callers), `jemalloc 5.3.0` (Facebook), and `mimalloc 2.1.7` (Microsoft). All four are loaded via `LD_PRELOAD` so the benchmark binary is identical across runs. Three benchmarks: `threadtest` (per-thread malloc/free pressure for scalability), `cache_thrash` (active false sharing), `cache_scratch` (passive false sharing).

Observations

Scalability. Through the per-socket physical-core boundary at 14 cores, all four allocators scale near-linearly. Hoard and `mimalloc` lead in absolute throughput; `jemalloc` has the best scaling efficiency (7.6× at 14 threads). At 8 threads Hoard delivers 765 Mops/s vs. `glibc`’s 307 Mops/s – a 2.5× speedup. Beyond 14 threads every allocator regresses sharply (`mimalloc` –77%, Hoard –67%, `jemalloc` –51%, `glibc` –42%) as threads spread across the second NUMA socket; at 56 threads `jemalloc` alone recovers above 400 Mops/s, consistent with its documented NUMA-aware design.

False sharing. On `cache_thrash`, all four allocators land within 1.5× of one another (`glibc` 18.77 Gops/s, Hoard 18.13, `mimalloc` 18.74, `jemalloc` 12.62) – a collapse of the paper’s headline 60× gap by roughly 40×. `perf c2c` confirmed the mechanism: under both `glibc` and Hoard, the profile captured *zero HITM events* (the textbook signature of false sharing) across ~370 samples

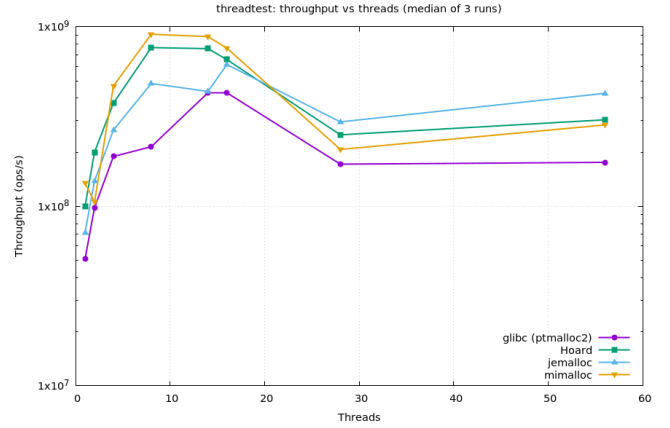


Figure 4. `threadtest` throughput vs. thread count, log-y, median of 3 runs per cell. The per-socket physical-core boundary (14) and the cross-socket NUMA cliff (28) are visible. Compare to Figure 3 of the original Hoard paper, which showed near-linear scaling through 14 CPUs vs. a flat `ptmalloc`.

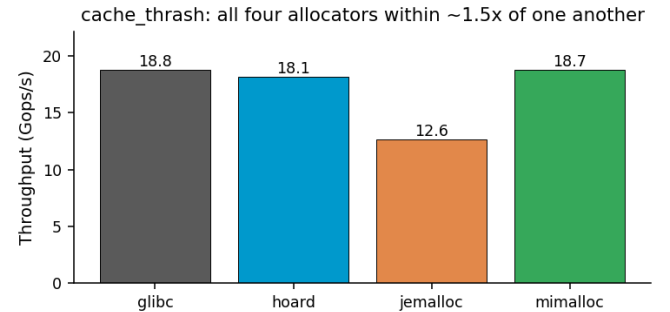


Figure 5. `cache_thrash` active-false-sharing benchmark at 8 threads. The paper measured up to 60× Hoard advantage over a 1999-era allocator; on 2026 Broadwell all four modern allocators land within 1.5× of one another.

each, despite hundreds of Store L1D misses. The cross-core coherence traffic that `cache_thrash` was specifically designed to provoke is absent.

Analysis

What the experiment shows. The qualitative claim reproduces clearly: thread-aware allocators do scale near-linearly through the available physical-core budget, where `glibc` lags by roughly 2–2.5×. The quantitative claim (60× on `cache_thrash`) does not.

What we cannot claim from the data. Our experiment is consistent with two distinct explanations for the smaller false-sharing gap on modern hardware that it does not separate. (a) `glibc` adopted per-thread arenas in 2.10 (2009), so the modern `ptmalloc2` we are comparing against is fundamentally a different allocator than the paper’s `libc-malloc`; the paper’s baseline simply does not exist on a modern Linux distribution. (b) Our `cache_thrash` workload may be too short for prefetcher warm-up effects to stabilize, or may be exercising a cache behavior different enough from the paper’s that the false sharing simply doesn’t trigger. The `perf c2c` zero-HITM

result rules out cross-core coherence as the active mechanism under glibc, but the deeper question of *why glibc no longer exhibits the pathology* (because of per-thread arenas, or because of something else like per-allocation cache-line padding) is not directly measured here.

The cleanest reading is a thesis about absorption: the paper’s idea (per-thread heaps to eliminate false sharing) won so completely that the system allocator we compare against now implements it. The differential measurement that motivated the paper has collapsed because the paper’s insight became mainstream.

5. Memory Layout as a Hidden Source of Performance Variance

The setup

Mytkowicz et al. argue that the same binary, compiled with the same flags, can produce measurably different runtimes when only its memory layout changes. The paper’s evidence (Figure 3, Table 1) is a sweep over linker padding and environment-variable size, with all other randomness sources controlled (ASLR off, dedicated machine, single-threaded benchmarks). They report runtime swings of 30–40% on SPEC INT 2006 workloads as a function of these layout changes alone.

SPEC INT 2006 is licensed and unavailable on our cluster, so we substitute the SQLite benchmark from Section 3 (smaller workload variant: 4 threads $\times$ 25,000 inserts, $\sim$ 1–2 s per run). The substitution is conservative for the paper’s central claim – Mytkowicz’s mechanism is layout-induced cache and TLB behavior, which is workload-agnostic so long as the workload has a non-trivial working set; SQLite’s prepared-statement INSERT loop has both a hot instruction path and a hot data path, so it should exhibit the layout sensitivity the paper describes if the mechanism is still present. Layout is varied via the size of the PADDING environment variable, which shifts initial stack alignment and indirectly all subsequent mappings. ASLR is disabled per-process via `setarch -R`. We swept 31 padding sizes from 0 to 4096 bytes in non-uniform steps, 10 runs per padding, and ran TMA (`perf stat -M`) and Cachegrind on the best- and worst-performing paddings.

Observations

Wall-clock. Best padding (1152 bytes) had mean elapsed 0.5957 s (± 0.063 , 95% CI); worst (1536 bytes) 0.7089 s (± 0.048). The 95% CIs do not overlap (best [0.533, 0.659] s, worst [0.661, 0.757] s), so the **19.0%** cross-padding gap is statistically significant despite per-padding CV of 8.7–20.2% on this shared multi-user machine. Variation is non-monotonic across padding sizes, exactly as in the paper’s figure.

TMA. A 5-run `perf stat -M Frontend_Bound, ..., Retiring` measurement at the best and worst paddings reproduced `Frontend_Bound` cleanly (0.30 vs. 0.31 – statistically indistinguishable), but the remaining top-down metrics could not all be read from a single PMU-counter group on this kernel/`perf` version, and we elected not to interleave runs

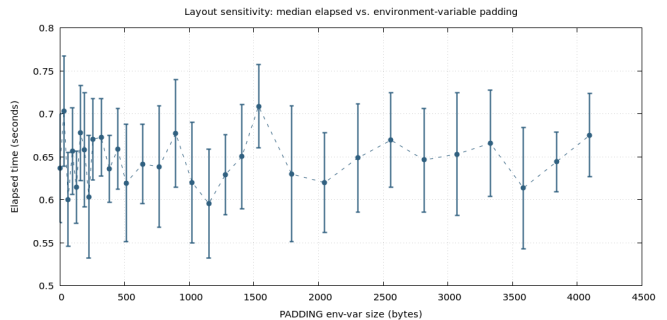


Figure 6. Median elapsed time vs. PADDING env-var size, 95% CI error bars, 10 runs per padding. The 19% gap between best (1152 B) and worst (1536 B) is statistically significant. Compare to Figure 3 of [3] for SPEC `perlbench`.

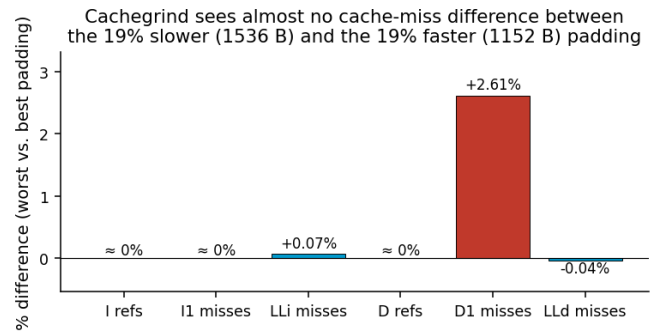


Figure 7. Cachegrind sees almost no cache-miss difference between the 19%-slower and 19%-faster paddings. Bars are percent change in worst vs. best; only D1 misses move appreciably (+2.6%). The mechanism the paper attributed the layout effect to (cache conflict misses) does not show up in our cachegrind run.

across separate counter groups because the per-padding wall-clock CV (8.7–20.2%) is large enough that across-run reassembly would not yield a credible signal at the 5-run sample size. *This is itself a reproduction-study lesson:* TMA assumes a counter-rich PMU and a `perf` build that exposes all events in one group; that assumption is increasingly fragile across kernel versions and is worth flagging in any methodology that depends on it.

Cachegrind. The most surprising result of this section is that **Cachegrind sees almost no cache-miss difference between the fast and slow paddings**, despite a 19% wall-clock gap on real hardware:

Metric	Best (1152)	Worst (1536)	Δ
I refs	1.152 G	1.152 G	−0.0001%
I1 misses	27,541 M	27,541 M	−0.001%
L1i misses	4,317	4,320	+0.07%
D refs	560.95 M	560.95 M	−0.0001%
D1 misses	104,837	107,576	+2.61%
LLd misses	48,261	48,242	−0.04%

Analysis

What the experiment shows. The paper’s qualitative claim reproduces strongly: layout matters at the non-overlapping-CI level, and a non-trivial fraction of what

practitioners report as “run-to-run noise” on benchmarks is deterministic layout effect. The paper’s quantitative claim is also approximately correct in order of magnitude (19% vs. 30–40%).

Where our data is too thin. Cachegrind ran exactly once per padding. Our 10-run sweep on real hardware shows per-padding CV of 8.7–20.2%. A single cachegrind run is not representative of a median, especially since under valgrind’s heavy instrumentation the workload’s timing characteristics change. The cachegrind elapsed times themselves – 13.0s and 12.9s respectively – are within each other’s noise floor under valgrind. The cache-miss *counts* are deterministic under cachegrind (the simulation is deterministic), so the near-equal miss counts are a real finding even if the elapsed-time inversion is not.

What theory says should be visible if the paper’s mechanism reproduces. The paper attributes its layout effect to cache conflict misses – pieces of frequently-accessed data landing in the same cache set on one layout but in different sets on another. Cachegrind’s deterministic LRU model is exactly the tool designed to count those misses. If conflict misses were dominant, we would expect to see a clear D1- or L1d-misses delta of comparable magnitude to the wall-clock gap. We don’t: D1 differs by 2.6%, L1d by $\approx 0\%$, I1 and L1i essentially zero.

What this leaves us with. The 19% wall-clock gap is real; the cache-conflict-miss explanation does not survive in our cachegrind data. Several mechanisms cachegrind cannot model could plausibly account for the gap: (1) hardware prefetchers (Broadwell has L1d, L2 stride prefetchers; layouts that match the prefetcher’s pattern produce identical *simulated* miss counts but very different real wall-clock times); (2) TLB pressure (cachegrind doesn’t model TLBs; layouts that shift across page boundaries change page-set spanning without changing cache-line set spanning); (3) branch-predictor BTB collisions (cachegrind by default doesn’t simulate the branch predictor).

Our experiment is consistent with multiple hypotheses (prefetchers, TLB pressure, BTB collisions) that it does not separate; we lack direct measurements distinguishing among them. What we claim is the negative result: *the specific mechanism the 2009 paper invoked is not the mechanism producing the effect on 2026 Broadwell, at least not in the form cachegrind can see.* The phenomenon migrated; the methodology (control for layout, sweep, measure variance) remains exactly the right way to measure it. A direct measurement of branch-predictor state (e.g., `perf stat -e branch-misses, BR_INST_RETIRED.NEAR_TAKEN` across paddings) would be the natural next experiment to disambiguate the BTB-collision hypothesis from the prefetcher and TLB hypotheses; we flag this as the most informative single follow-up. Subsequent work by Curtsinger and Berger [7] arrives at the same conclusion via a different route: their *Stabilizer* system randomizes layout repeatedly at runtime, validating that the layout-sensitivity phenomenon Mytkowicz identified is robust enough to be

the basis of a statistically sound benchmarking methodology.

6. Three Modes of Decay

The three reproductions converge on a single observation about what happens to performance findings over decades. *The specific quantitative claims in performance papers are statements about a particular hardware/software/workload triple, and they decay; the methodologies that produced those claims are comparatively durable.* The mode of decay differs across our three papers, and the differences are themselves informative.

Engineered out. Coz’s 2015 mutex bottleneck shrank by roughly $5\times$ in absolute cost between glibc 2.17 and glibc 2.35: the ratio of work-per-mutex to work-per-iteration of the inner loop became small enough that the saving from optimizing the mutex disappeared into the noise. The mainline implementation did the work the paper recommended, and then some. The paper’s *methodology* (causal profiling) still correctly identifies that the line is no longer a useful target.

Absorbed into the mainline. Hoard’s per-thread arena design moved from a 2000 research prototype into glibc 2.10 in 2009. The differential measurement that originally motivated the paper has collapsed because what the paper compared against (a single-arena `libc-malloc`) no longer exists in any production environment. The paper’s *methodology* (per-thread heaps with superblock ownership) became the standard implementation strategy and is now used by every major modern allocator.

Mechanism relocated. Mytkowicz’s layout-induced variance persists at reduced magnitude (19% vs. 30–40%) but the simple cache-conflict-miss mechanism the paper attributed it to does not reproduce on Broadwell with its much-larger LLC. The phenomenon has likely migrated into hardware features (prefetchers, TLBs, BTBs) that 2009-era simulation tools were not designed to model. The paper’s *methodology* (control for layout or report variance over many layouts) is more important than ever: practitioners running benchmarks with default ASLR are unknowingly averaging deterministic layout effects into their “noise.”

The three modes are not the same outcome. “Engineered out” means the bottleneck is gone. “Absorbed” means the bottleneck is gone because the paper’s fix became standard. “Relocated” means the phenomenon is still there but moved to where current tools can’t see it. A reproduction study a decade later is the substantive way to discover *which* of these has happened to a given paper’s claim.

7. Conclusion

What we measured agrees with each paper’s qualitative claim and disagrees with each paper’s quantitative claim, in three distinct ways tied to three distinct mechanisms of decay. The honest framing of a reproduction study is not “the paper holds up” or “the paper failed to replicate”; it is a map of which specific claims survived which specific changes in the underlying stack, and a tally of which ones

we lacked the measurement precision to fully resolve.

A practical lesson cuts across all three studies: when reading performance papers, separate the methodological contribution (“causal profiling identifies counterfactual bottlenecks”, “per-thread arenas eliminate false sharing”, “layout shifts deterministically affect runtime”) from the quantitative finding (“+25%”, “60×”, “+30–40%”). The former travels across decades; the latter is a snapshot. Reproducing classic results is, at heart, an audit of which of those snapshots is still in focus.

References

- [1] Charlie Curtsinger and Emery D. Berger. *Coz: Finding Code that Counts with Causal Profiling*. SOSP, 2015.
- [2] Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, and Paul R. Wilson. *Hoard: A Scalable Memory Allocator for Multithreaded Applications*. ASPLOS, 2000.
- [3] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. *Producing Wrong Data Without Doing Anything Obviously Wrong!* ASPLOS, 2009.
- [4] Herb Sutter. *The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software*. Dr. Dobbs’s Journal, March 2005.
- [5] Ulrich Drepper. *What Every Programmer Should Know About Memory*. Red Hat technical report, November 2007.
- [6] Ulrich Drepper. *Futexes Are Tricky*. Red Hat technical report, May 2011.
- [7] Charlie Curtsinger and Emery D. Berger. *Stabilizer: Statistically Sound Performance Evaluation*. ASPLOS, 2013.